

Atividade Prática 2 – Introdução à placa DE10-Lite

Pedro Lucas dos Reis Silva - RA: 2272040

02/04/2026

1 Introdução

Esta atividade tem como objetivo a introdução à placa FPGA DE10-Lite, realizando a implementação de circuitos combinacionais em hardware real. A atividade é composta por dois exercícios:

1. Implementação das 8 portas lógicas (conforme Atividade 1) na placa DE10-Lite, utilizando chaves (slide switches) como entradas e LEDs como saídas.
2. Projeto de um circuito de controle para uma máquina de cópias, que detecta quando duas ou mais chaves estão fechadas simultaneamente, utilizando simplificação por mapa de Karnaugh.

A placa DE10-Lite utiliza o FPGA Intel MAX 10 e possui 10 slide switches (SW0–SW9) e 10 LEDs vermelhos (LEDR0–LEDR9), operando em nível lógico 3.3V LVTTTL. O mapeamento entre os pinos lógicos do VHDL e os pinos físicos da placa é feito através do *Pin Planner* do Quartus Prime.

Os códigos VHDL completos encontram-se nos Apêndices ao final deste relatório.

2 Exercício 1 – Portas Lógicas na DE10-Lite

2.1 Descrição

Neste exercício, o mesmo circuito desenvolvido na Atividade 1 (8 portas lógicas: NOT a, NOT b, AND, OR, NAND, NOR, XOR, XNOR) foi implementado na placa DE10-Lite. As entradas a e b foram associadas às slide switches SW0 e SW1, e cada saída (z1 a z8) foi associada a um LED.

2.2 Pin Planner

A Tabela 1 apresenta o mapeamento dos pinos utilizado no Quartus Prime para o Exercício 1.

Tabela 1: Pin Planner – Exercício 1 (portas lógicas).

Sinal	Componente	Pino FPGA	I/O Standard
a	SW0 (Slide Switch 0)	PIN_C10	3.3-V LVTTTL
b	SW1 (Slide Switch 1)	PIN_C11	3.3-V LVTTTL
z1–z8	LEDR0–LEDR7	—	3.3-V LVTTTL

Top View - Wire Bond MAX 10 - 10M50DAF484I7G

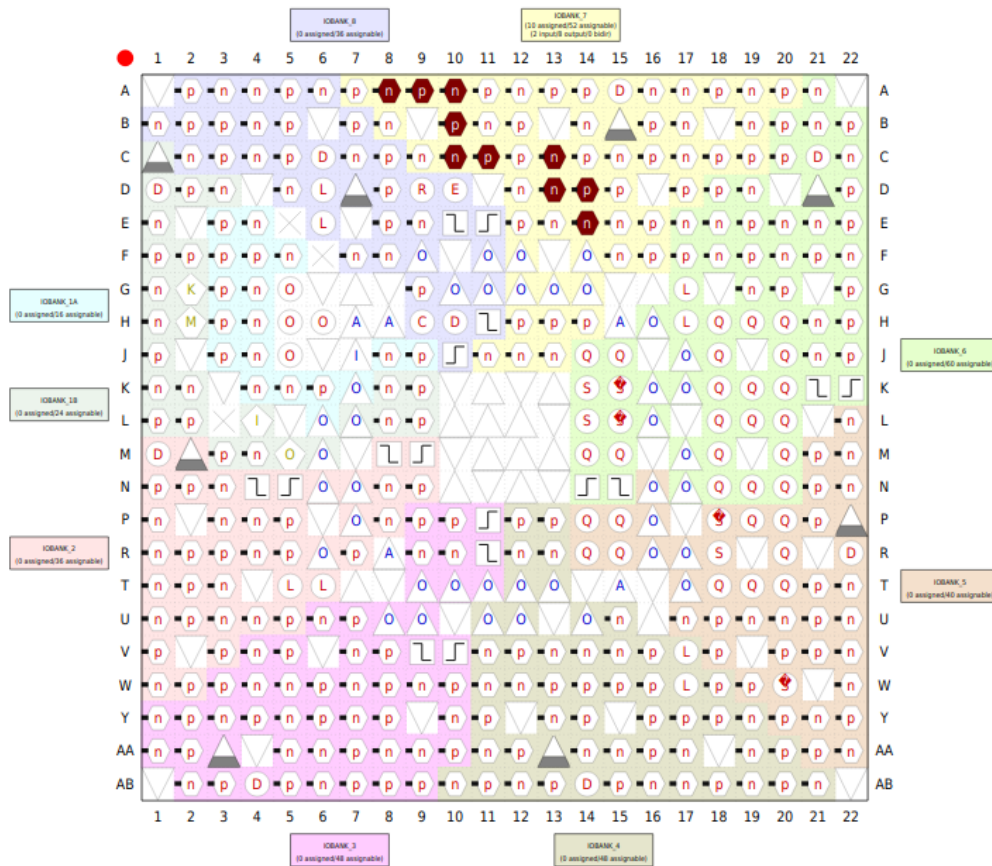


Figura 1: Pin Planner do Exercício 1 no Quartus Prime.

3 Exercício 2 – Circuito de Controle da Máquina de Cópias

3.1 Descrição do Problema

Quatro chaves (SW1 a SW4) estão posicionadas ao longo do caminho do papel em uma máquina de cópias. Cada chave é normalmente aberta e fecha quando o papel passa por ela. Cada chave possui um resistor de *pull-up* conectado a +5V, garantindo nível lógico alto quando aberta e nível baixo quando fechada. Na placa DE10-Lite, as slide switches já possuem circuito de pull-up integrado operando em 3.3V LVTTTL, de modo que a lógica permanece a mesma: chave ativada = '1', chave desativada = '0'.

O circuito deve produzir saída alta sempre que **duas ou mais** chaves estejam fechadas ao mesmo tempo. Como restrição do problema, as chaves SW1 e SW4 **nunca** estarão fechadas simultaneamente, o que permite o uso de condições *don't care* para simplificação.

3.2 Tabela Verdade

A Tabela 2 apresenta a tabela verdade completa. As combinações onde SW1=1 e SW4=1 simultaneamente são marcadas como *don't care* (X), pois esse cenário é fisicamente impossível.

Tabela 2: Tabela verdade do circuito de controle – saída $z=1$ quando 2 ou mais chaves fechadas.

SW1	SW2	SW3	SW4	z	Observação
0	0	0	0	0	Nenhuma chave fechada
0	0	0	1	0	Apenas 1 chave
0	0	1	0	0	Apenas 1 chave
0	0	1	1	1	SW3 + SW4
0	1	0	0	0	Apenas 1 chave
0	1	0	1	1	SW2 + SW4
0	1	1	0	1	SW2 + SW3
0	1	1	1	1	SW2 + SW3 + SW4
1	0	0	0	0	Apenas 1 chave
1	0	0	1	X	<i>don't care</i> (SW1+SW4 impossível)
1	0	1	0	1	SW1 + SW3
1	0	1	1	X	<i>don't care</i>
1	1	0	0	1	SW1 + SW2
1	1	0	1	X	<i>don't care</i>
1	1	1	0	1	SW1 + SW2 + SW3
1	1	1	1	X	<i>don't care</i>

3.3 Mapa de Karnaugh

A tabela verdade foi transposta para o mapa de Karnaugh de 4 variáveis, com SW1·SW2 nas linhas e SW3·SW4 nas colunas (ordenação Gray):

Tabela 3: Mapa de Karnaugh – agrupamentos para simplificação.

SW1·SW2	SW3·SW4			
	00	01	11	10
00	0	0	1	0
01	0	1	1	1
11	1	X	X	1
10	0	X	X	1

Os agrupamentos identificados (incluindo *don't cares* para maximizar os grupos) são:

1. **SW3·SW4**: cobre as células (00,11), (01,11), (11,11), (10,11)
2. **SW2·SW4**: cobre as células (01,01), (01,11), (11,01), (11,11)
3. **SW2·SW3**: cobre as células (01,10), (01,11), (11,10), (11,11)
4. **SW1·SW2**: cobre as células (11,00), (11,01), (11,10), (11,11)
5. **SW1·SW3**: cobre as células (10,10), (10,11), (11,10), (11,11)

3.4 Equação Booleana Simplificada

A equação mínima em soma de produtos obtida é:

$$z = SW1 \cdot SW2 + SW1 \cdot SW3 + SW2 \cdot SW3 + SW2 \cdot SW4 + SW3 \cdot SW4 \quad (1)$$

Note que o termo $SW1 \cdot SW4$ não aparece na equação, pois essa combinação é fisicamente impossível (condição *don't care*) e foi aproveitada para simplificar os demais agrupamentos.

3.5 Implementação VHDL

A equação (1) foi implementada diretamente em VHDL no estilo *dataflow*, com uma única atribuição concorrente (ver Apêndice B). Cada termo da equação corresponde a um par de chaves que, quando ambas fechadas, ativa a saída.

3.6 Pin Planner

A Tabela 4 apresenta o mapeamento dos pinos utilizado no Quartus Prime para o Exercício 2.

Tabela 4: Pin Planner – Exercício 2 (máquina de cópias).

Sinal	Componente	Pino FPGA	I/O Standard
chave1 (SW1)	SW0 (Slide Switch 0)	PIN_C10	3.3-V LVTTL
chave2 (SW2)	SW1 (Slide Switch 1)	PIN_C11	3.3-V LVTTL
chave3 (SW3)	SW2 (Slide Switch 2)	PIN_D12	3.3-V LVTTL
chave4 (SW4)	SW3 (Slide Switch 3)	PIN_C12	3.3-V LVTTL
z (saída)	LEDR0 (LED Vermelho)	PIN_A8	3.3-V LVTTL

Top View - Wire Bond MAX 10 - 10M50DAF484I7G

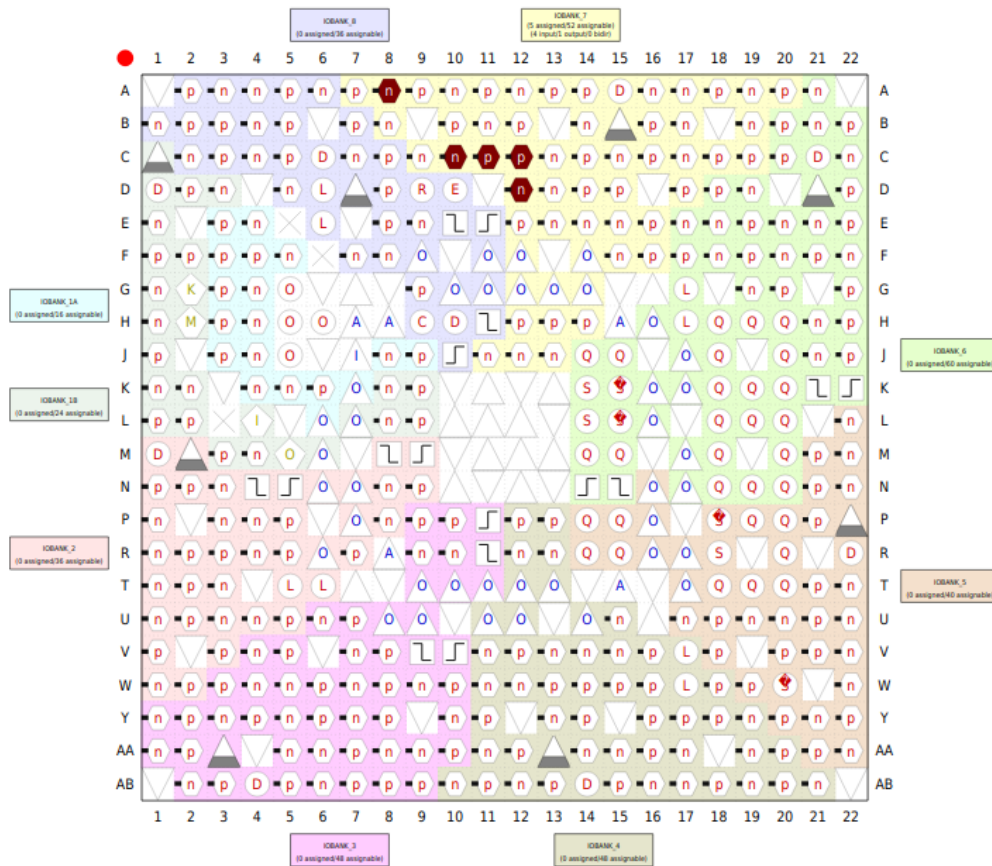


Figura 2: Pin Planner do Exercício 2 no Quartus Prime.

3.7 Simulação

A simulação foi realizada no Quartus Prime, testando todas as 16 combinações possíveis das 4 chaves. A Figura 3 apresenta as formas de onda obtidas. Observa-se que a saída *z* vai para nível alto somente quando duas ou mais chaves estão ativadas.

Nas combinações onde *SW1*=1 e *SW4*=1 simultaneamente (condições *don't care*), a saída também resulta em nível alto. Isso é esperado: no mapa de Karnaugh, essas condições foram tratadas como '1' para permitir agrupamentos maiores e obter uma equação mais simples. Como essa combinação de entrada é fisicamente impossível na máquina de cópias, o valor da saída nesse caso é irrelevante — poderia ser 0 ou 1 sem afetar o funcionamento real do circuito.

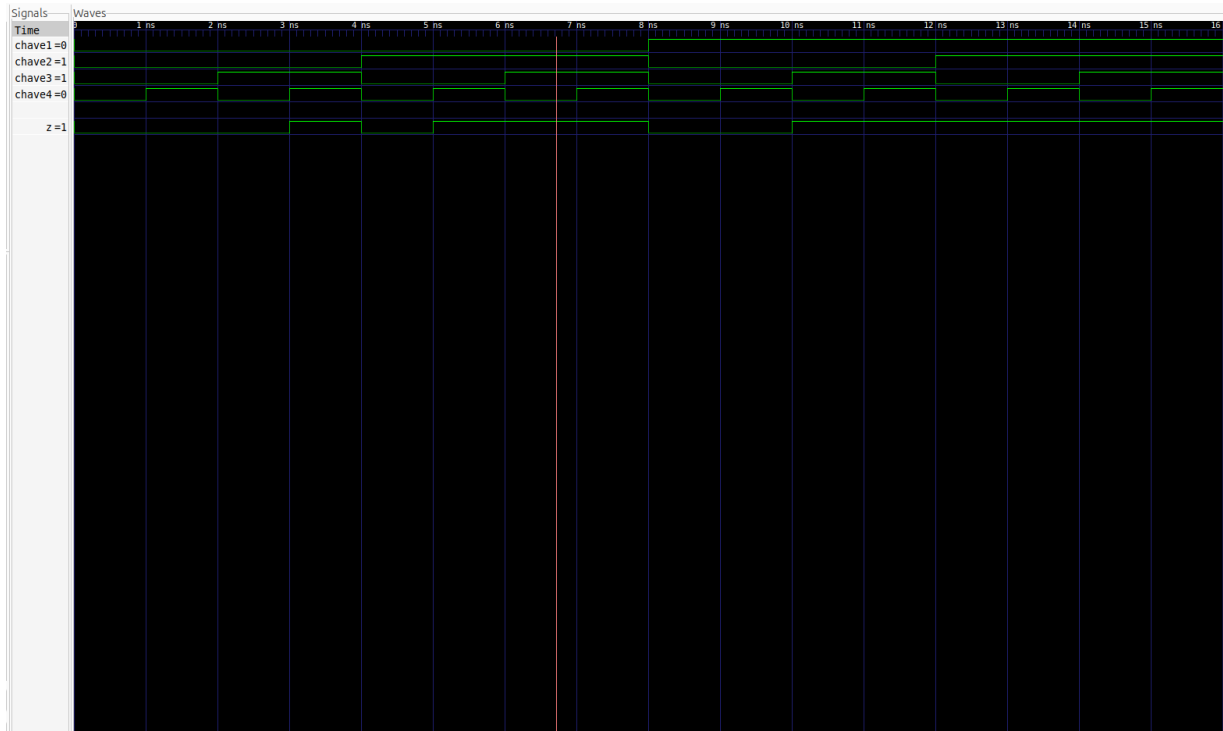


Figura 3: Simulação do circuito de controle da máquina de cópias no Quartus Prime.

3.8 Diagrama RTL

O diagrama RTL foi gerado pelo Quartus Prime através da ferramenta *RTL Viewer* (Tools → Netlist Viewers → RTL Viewer). A Figura 4 apresenta a estrutura do circuito sintetizado, onde se observam as portas AND de 2 entradas e a porta OR combinando os 5 termos da equação simplificada.

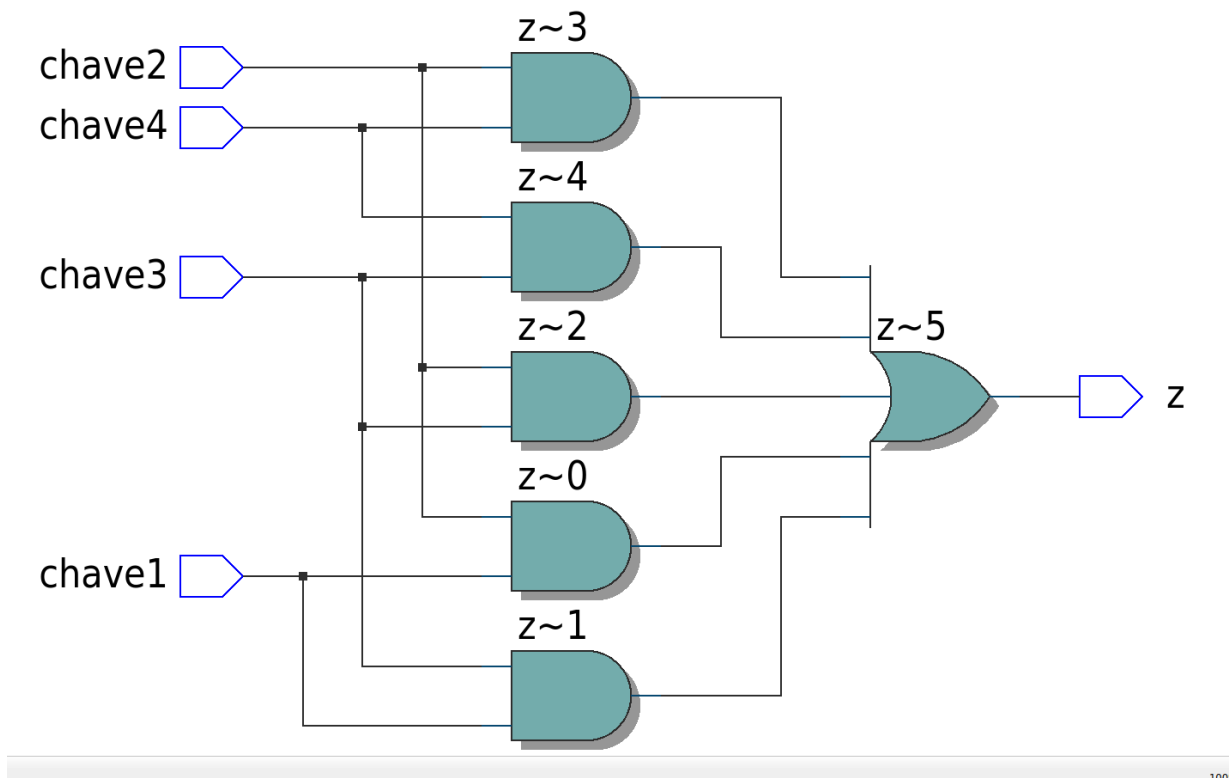


Figura 4: Diagrama RTL do circuito de controle, gerado pelo Quartus Prime.

4 Considerações Finais

Ambos os exercícios foram implementados com sucesso na placa DE10-Lite. No Exercício 1, as 8 portas lógicas responderam corretamente às combinações de entrada através das slide switches, com os LEDs indicando os resultados esperados. No Exercício 2, o circuito de controle da máquina de cópias detectou corretamente quando duas ou mais chaves estavam fechadas, acendendo o LED correspondente.

O uso do mapa de Karnaugh no Exercício 2 permitiu obter uma equação simplificada com 5 termos de produto, aproveitando as condições *don't care* da restrição $SW1 \cdot SW4 = 0$ para reduzir a complexidade do circuito.

O Quartus Prime mostrou-se a ferramenta adequada para esta etapa do desenvolvimento, realizando a síntese, o mapeamento de pinos e a gravação na placa FPGA. A simulação e a geração do diagrama RTL foram realizadas diretamente no Quartus. Para as atividades anteriores, a simulação foi feita com ferramentas open source (GHDL + GTKWave) em ambiente Linux, o que se mostrou viável para verificação funcional antes da síntese no Quartus.

A Código VHDL – Exercício 1 (Portas Lógicas)

```

1  -- =====
2  -- Atividade 2 - Exercício 1: Portas Logicas na DE10-Lite
3  -- Pin Planner: a=SW0(PIN_C10), b=SW1(PIN_C11), z1..z8=LEDR
4  -- =====
5  library ieee;
6  use ieee.std_logic_1164.all;
7
8  entity atividade1 is
9      port (
10         a      : in  bit;    -- SW0: entrada A
11         b      : in  bit;    -- SW1: entrada B
12         z1     : out bit;    -- LEDR0: NOT a
13         z2     : out bit;    -- LEDR1: NOT b
14         z3     : out bit;    -- LEDR2: AND
15         z4     : out bit;    -- LEDR3: OR
16         z5     : out bit;    -- LEDR4: NAND
17         z6     : out bit;    -- LEDR5: NOR
18         z7     : out bit;    -- LEDR6: XOR
19         z8     : out bit;    -- LEDR7: XNOR
20     );
21 end entity;
22
23 architecture dataflow of atividade1 is
24 begin
25     z1 <= not a;             -- inversao de A
26     z2 <= not b;             -- inversao de B
27     z3 <= a and b;           -- AND
28     z4 <= a or b;            -- OR
29     z5 <= a nand b;          -- NAND
30     z6 <= a nor b;           -- NOR
31     z7 <= a xor b;           -- XOR
32     z8 <= a xnor b;          -- XNOR
33 end architecture;

```

Listing 1: Arquivo atividade1.vhd – 8 portas lógicas na DE10-Lite.

B Código VHDL – Exercício 2 (Máquina de Cópias)

```

1  -- =====
2  -- Atividade 2 - Exercício 2: Circuito de controle maquina de copias
3  -- Saida HIGH quando 2 ou mais chaves estao fechadas.
4  -- SW1 e SW4 nunca fecham ao mesmo tempo (don't care).
5  --
6  -- Equacao (Karnaugh):
7  --  $z = SW1.SW2 + SW1.SW3 + SW2.SW3 + SW2.SW4 + SW3.SW4$ 
8  --
9  -- Pin Planner (DE10-Lite):
10 -- chave1=SW0(PIN_C10), chave2=SW1(PIN_C11),
11 -- chave3=SW2(PIN_D12), chave4=SW3(PIN_C12),
12 -- z=LEDRO(PIN_A8)
13 -- =====
14 library ieee;
15 use ieee.std_logic_1164.all;
16
17 entity atividade2 is
18     port (
19         chave1 : in  bit;  -- SW0: chave 1
20         chave2 : in  bit;  -- SW1: chave 2
21         chave3 : in  bit;  -- SW2: chave 3
22         chave4 : in  bit;  -- SW3: chave 4
23         z       : out bit  -- LEDRO: saida
24     );
25 end entity;
26
27 architecture dataflow of atividade2 is
28 begin
29     -- Equacao simplificada via Karnaugh
30     -- (todas as combinacoes de 2 chaves, exceto
31     -- SW1.SW4 que e don't care)
32     z <= (chave1 and chave2) or
33         (chave1 and chave3) or
34         (chave2 and chave3) or
35         (chave2 and chave4) or
36         (chave3 and chave4);
37 end architecture;

```

Listing 2: Arquivo atividade2.vhd – circuito de controle da máquina de cópias.

C Resolução Manuscrita – Tabela Verdade e Mapa de Karnaugh

A Figura 5 apresenta a resolução manuscrita do Exercício 2, contendo a tabela verdade completa com as condições *don't care*, o mapa de Karnaugh em ordenação Gray com os agrupamentos identificados, e a equação booleana simplificada obtida.

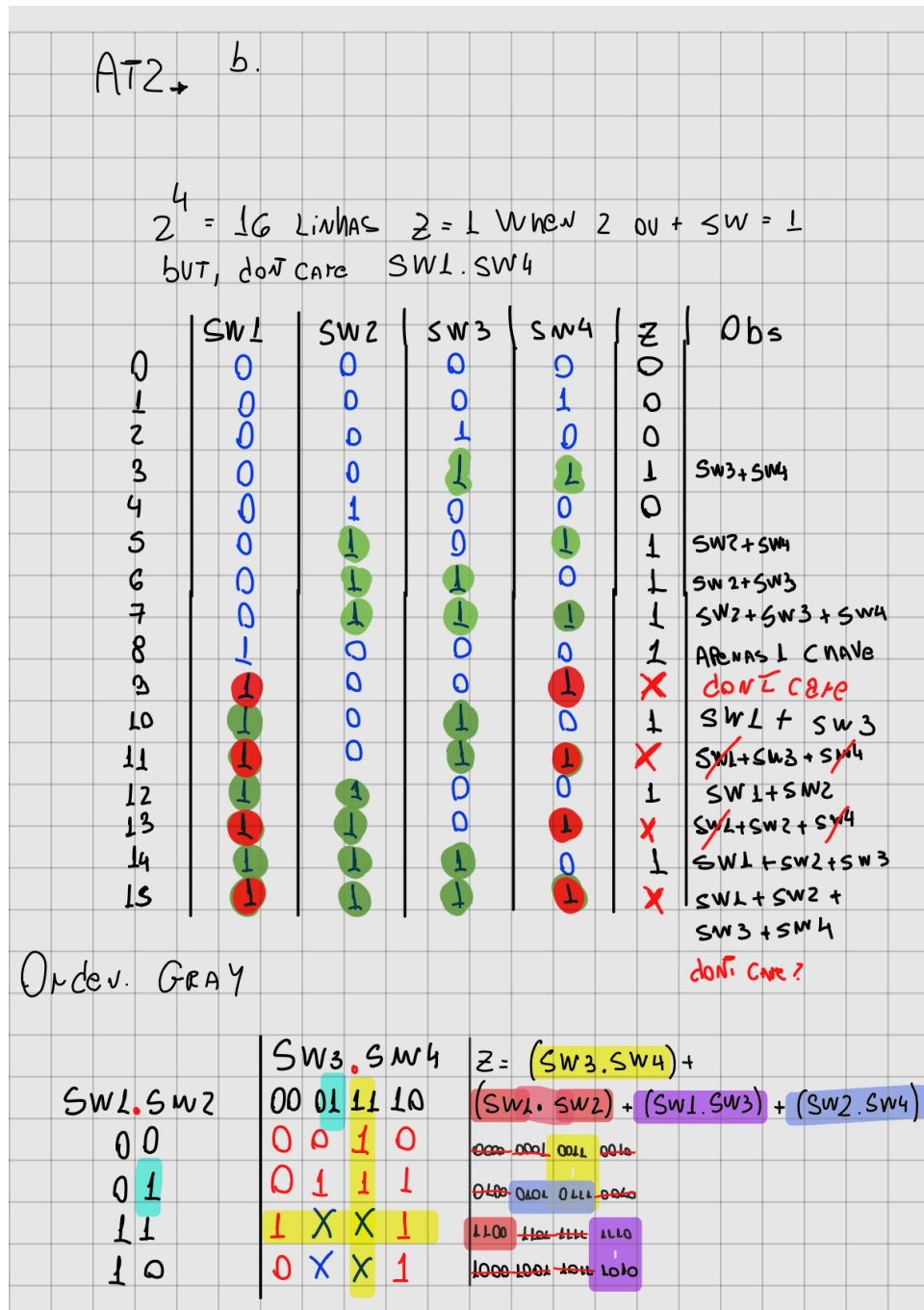


Figura 5: Resolução manuscrita: tabela verdade, mapa de Karnaugh e agrupamentos.

D Código VHDL – Testbench do Exercício 2

```

1  -- Testbench para atividade2: circuito de controle
2  -- Testa todas as 16 combinacoes de 4 chaves
3
4  entity atividade2_tb is
5  end entity;
6
7  architecture tb of atividade2_tb is
8      signal chave1, chave2, chave3, chave4 : bit;
9      signal z : bit;
10 begin
11     uut: entity work.atividade2
12         port map(
13             chave1 => chave1, chave2 => chave2,
14             chave3 => chave3, chave4 => chave4,
15             z => z
16         );
17
18     process
19     begin
20         chave1<='0'; chave2<='0'; chave3<='0'; chave4<='0'; wait for 1 ns;
21         chave1<='0'; chave2<='0'; chave3<='0'; chave4<='1'; wait for 1 ns;
22         chave1<='0'; chave2<='0'; chave3<='1'; chave4<='0'; wait for 1 ns;
23         chave1<='0'; chave2<='0'; chave3<='1'; chave4<='1'; wait for 1 ns;
24         chave1<='0'; chave2<='1'; chave3<='0'; chave4<='0'; wait for 1 ns;
25         chave1<='0'; chave2<='1'; chave3<='0'; chave4<='1'; wait for 1 ns;
26         chave1<='0'; chave2<='1'; chave3<='1'; chave4<='0'; wait for 1 ns;
27         chave1<='0'; chave2<='1'; chave3<='1'; chave4<='1'; wait for 1 ns;
28         chave1<='1'; chave2<='0'; chave3<='0'; chave4<='0'; wait for 1 ns;
29         chave1<='1'; chave2<='0'; chave3<='0'; chave4<='1'; wait for 1 ns;
30         chave1<='1'; chave2<='0'; chave3<='1'; chave4<='0'; wait for 1 ns;
31         chave1<='1'; chave2<='0'; chave3<='1'; chave4<='1'; wait for 1 ns;
32         chave1<='1'; chave2<='1'; chave3<='0'; chave4<='0'; wait for 1 ns;
33         chave1<='1'; chave2<='1'; chave3<='0'; chave4<='1'; wait for 1 ns;
34         chave1<='1'; chave2<='1'; chave3<='1'; chave4<='0'; wait for 1 ns;
35         chave1<='1'; chave2<='1'; chave3<='1'; chave4<='1'; wait for 1 ns;
36         wait;
37     end process;
38 end architecture;

```

Listing 3: Arquivo atividade2_tb.vhd – testbench com todas as 16 combinações.